



Oracle EBS R12 Online Bank Payments Integration

Reliance Weaving Mills Ltd

SCOPE

Some key aspects that typically fall within the scope of this project:

- **Bank Integration:** The project scope includes integrating Oracle EBS R12 with one or more banks for processing online payments. This involves setting up communication channels, configuring security protocols, and ensuring compatibility with the bank's systems.
- **Payment Methods:** Define and configure payment methods supported by the banks, such as Inter Bank Fund Transfer (IBFT), Corporate Cheque (CCHQ), Salary Transfer, or other electronic payment formats.
- **Payment Formats:** Configure payment formats within Oracle EBS R12 to generate payment files that comply with the requirements of the banks. This may involve customizing standard payment formats or creating new ones based on bank specifications.
- **Data Mapping:** Map payment data fields within Oracle EBS R12 to match the format expected by the banks. Ensure that all required information, such as payment amounts, beneficiary details, and reference numbers, are accurately included in the payment files.

Benefits

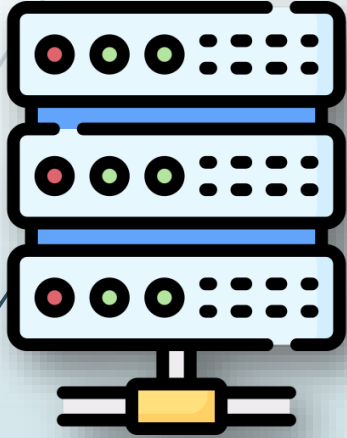
- **Efficiency:** Manual payment processing can be time-consuming and error-prone. Integrating online bank payments automates the process, saving time and reducing the risk of errors associated with manual data entry.
- **Accuracy:** Automation improves the accuracy of payment processing by reducing the likelihood of human errors. By integrating directly with banks, Oracle EBS R12 ensures that payment data is transmitted accurately and securely, minimizing discrepancies in financial transactions.
- **Cost Savings:** Automated payment processing can lead to cost savings by reducing the need for manual labor and streamlining financial operations. Organizations can also avoid costly errors and penalties associated with manual payment processing.
- **Real-time Processing:** Integration enables real-time processing of payments, providing faster access to funds and improving cash flow management. This is particularly beneficial for organizations that require timely payments to suppliers or vendors.
- **Enhanced Security:** Integrating online bank payments with Oracle EBS R12 allows organizations to implement robust security measures to protect sensitive financial data. Encryption, authentication, and other security features help safeguard payment information during transmission and storage.

Benefits

- **Improved Visibility and Reporting:** Integration provides better visibility into payment activities by centralizing payment data within Oracle EBS R12. This enables organizations to generate accurate financial reports, monitor payment trends, and track transaction histories more effectively.
- **Enhanced Vendor Relationships:** Automating payment processing can lead to faster and more reliable payments to suppliers and vendors. This can improve relationships with business partners and suppliers by ensuring timely payments and reducing payment-related disputes.
- **Scalability:** Integrating online bank payments with Oracle EBS R12 allows organizations to scale their payment processing capabilities as their business grows. Additional banks, payment methods, and transaction volumes can be accommodated within the integrated system.
- **Paperless Environment:** This project is considered as another milestone towards the Paperless Environment Drive initiated at Fatima Group. After maturity and confidence building, this project will help a great deal in reduction of paper used in payment processes.

Initial Infrastructure

Reliance Prod Server
IP: x.x.x.x

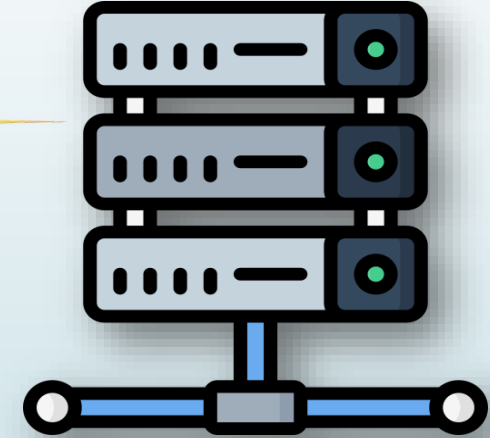


OS Oracle Linux 6

WAN LINK

Bank SFTP Server

Bank Bank Portal
Server

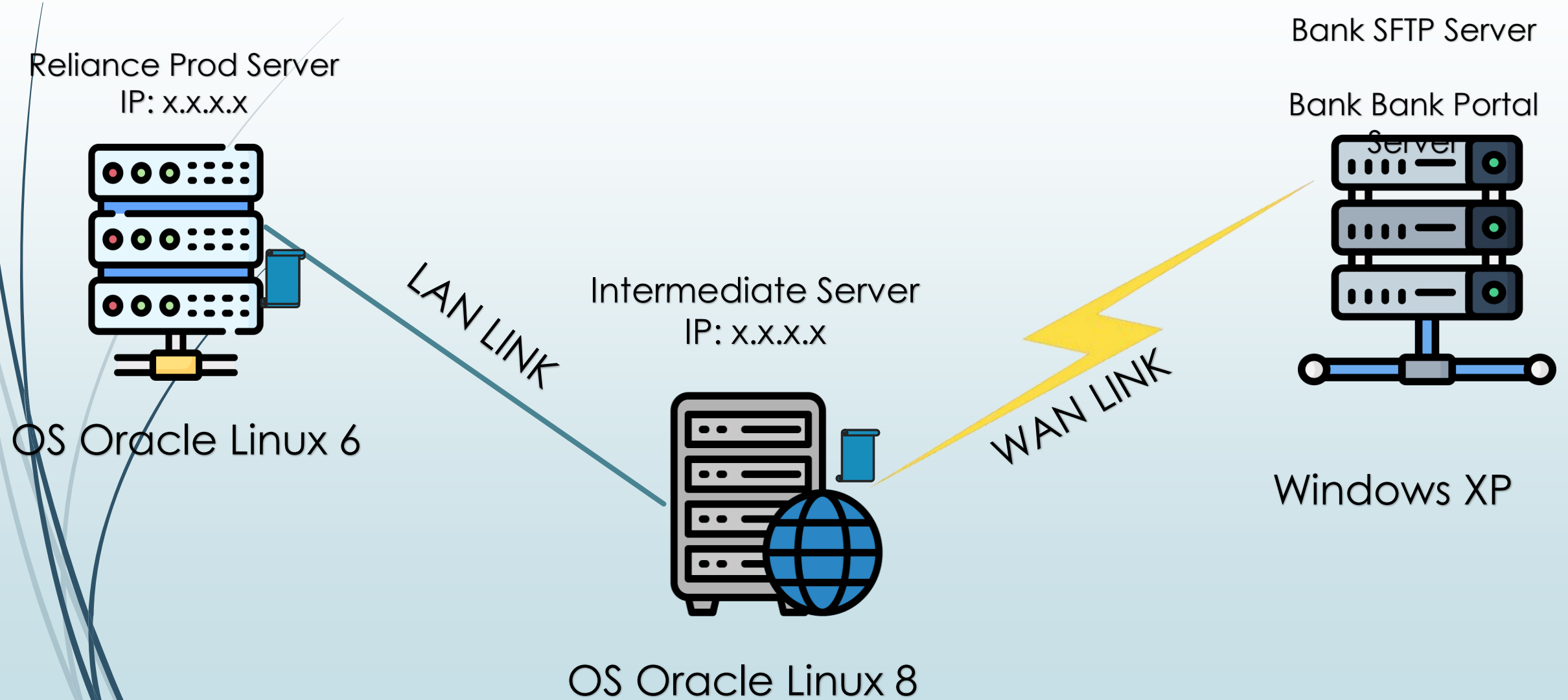


Windows XP/2008

Network Configurations

- To make our infrastructure more secure we have added an intermediate server (banks.Fatima-group.com) between our RWML Prod and BANK's sftp server so that our Prod is not exposed to external traffic. Which acts another layer of protection from potential cyber attacks like ransomware, spyware, malware and other viruses.
- Only intermediate server is allowed to communicate with banks by mapping its local IP on our Live IPs for WAN communication.
- Network File Sharing (NFS) protocol is used to mount network shared drive on the Prod server for file/cheques sharing.
- Intermediate server is restricted to transfer files on the Bank's portal not back to PROD server.

Secure Infrastructure



Trust Relationship and Handshaking

- We have used **SSH V2** protocol to establish secure connectivity with the Bank SFTP Server. Secure File Transfer (**SFTP**) protocol is used for file sharing using default port 22 of sftp.
- This method includes Public-Private Key sharing, Where Fatima Group has shared it's Public key with Bank for trust relationship. When connection is initiated we are prompted to enter our private key which is not shared with anyone. A successful connection is established and communication is allowed.
- After testing, a script is coded to automate the process so that manual entry of passwords is avoided.
- Several other scripts are written to automate the whole process so that the job becomes real time and error free.
- A script is placed on the Reliance Production server which enables the server to push any new file/ cheque created. This script is scheduled to run after every 5 minutes. A copy of transferred files is kept at Production Server locally.
- Other scripts are placed on the intermediate server that establishes secure connection with the Bank portal, checks for files received from the Production Server and transfers them to the Bank Portal. A copy of transferred files is maintained at this sever and file names of cheques transferred are also kept locally.

Benefits of Automation

➤ Streamlined Payment Processing:

- **Batch Payment Processing:** Automate the processing of multiple payments in batches, reducing the time required to handle each transaction manually.
- **Scheduled Payments:** Schedule scripts to execute at specific times, ensuring timely payment processing and avoiding delays.

➤ Error Reduction:

- **Validation Checks:** Automate validation of payment details (e.g., account numbers, amounts) before processing to reduce errors.
- **Consistency:** Ensure that the same steps are followed for each transaction, minimizing the risk of human error.

➤ Improved Security:

- **Encryption and Secure Transfer:** Automate the encryption of sensitive payment information and secure transfer protocols to protect data integrity and confidentiality.
- **Access Controls:** Use scripts to enforce strict access controls, ensuring that only authorized personnel can initiate or modify payment processes.

Benefits of Automation

➤ Increased Efficiency:

- **Real-time Processing:** Automate real-time processing of transactions, providing faster service to customers.
- **Resource Optimization:** Free up human resources to focus on exception handling and more complex tasks that require human intervention.

➤ Scalability:

- **Handling High Volume Transactions:** Scripts can be designed to handle high volumes of transactions efficiently, accommodating growing business needs.
- **Adaptability:** Easily modify scripts to integrate with new banking APIs or payment gateways as needed.

➤ Enhanced Reporting and Auditing:

- **Transaction Logs:** Automatically generate detailed logs of all transactions, providing a clear audit trail for compliance and reporting purposes.
- **Automated Reports:** Schedule and generate regular reports on payment statuses, successes, and failures.

Script on RWML Prod Server

- We have created a script on Production server which will copy the payment files from local directories (CCHQ, IBFT, LFT etc.) to Intermediate Server in corresponding directories (CCHQ, IBFT, LFT etc.).
- This script will also keep a copy of transferred files on the Production Server in other directories.
- Lastly, Real time synchronization also known as **rsync** tool is used in the script to make sure that exact copies of files are kept on Production and Intermediate Server. If there is a glitch or any abnormality during the transfer of files or lets say if a file is missed on the first attempt the production server will move it to the destination and vice versa.
- A copy of script is also attached in next slide for understanding.

Script

```
#!/bin/bash

# Array of source directories
source_dirs=(
    "/opt/d02/bank_integration/payments/HBL/IBFT"
    "/opt/d02/bank_integration/payments/HBL/payment_summary"
    "/opt/d02/bank_integration/payments/HBL/salary"
    "/opt/d02/bank_integration/payments/HBL/LFT"
    "/opt/d02/bank_integration/payments/HBL/SALARY_TRANSFER"
    "/opt/d02/bank_integration/payments/HBL/SMART_CHEQUE"
    "/opt/d02/bank_integration/payments/HBL/CCHQ"
)

# Destination directory on the NFS share
destination_directory="/exp/rwml/HBL/"

# Local directory for storing copied files
copied_directory="/banks/HBL/"

# Loop through each source directory
for source_dir in "${source_dirs[@]}; do
    # Get the subdirectory name
    sub_dir=${source_dir##*/}

    # Copy files to the destination directory
    rsync -avz "$source_dir"/*.txt "$destination_directory/$sub_dir/"

# Sync destination directory back to source directories
for source_dir in "${source_dirs[@]}; do
    sub_dir=${source_dir##*/}
    rsync -avz "$destination_directory/$sub_dir/" "$source_dir"
done
```

Script on Intermediate Server

- We established connection with the **Bank sftp** server using **ssh v2** protocol which is responsible for secure connectivity by using **key pair** (public-private). Public key of our server is shared with the bank's server. When a connection request is made from our side, request for private key is created by the Bank server. Upon entering the private key, connection is established with the Bank sftp server.
- To automate the process and avoid human interaction for connectivity, a script (**expect**) is placed on the intermediate server which upon trigger connects with the Bank sftp server and passes the private key and password itself. Thus making the process secure and error free.
- Expect script is placed with in another script (**inotify**), inotify is a real time monitoring tool. This tool monitors the directories 24*7, if there is a change or a new file is created in the monitored directories it triggers the expect script which establishes connection with the Bank sftp server and places the new file in corresponding directory on the target server.
- Further there is one other functionality of the script, a list of names of uploaded files is kept in corresponding files. When the inotify script triggers, it checks the uploaded files list. If the file is already uploaded it will bypass the expect script and ends the process but if the filename is not present in the uploaded files list, it will trigger the expect and new files will be copied on the Bank sftp server portal.


```
#!/bin/bash
```

```
# Set variables
```

```
host="secureftp.[REDACTED]"
```

```
username="re[REDACTED]1"
```

```
passphrase="[REDACTED]"
```

```
password="[REDACTED]"
```

```
local_root="/opt/data/nfshare/rwml/HBL"
```

```
backup_root="/opt/backup/rwml/HBL" # Backup directory outside local root
```

```
remote_root="" # Specify the remote root directory here
```

```
# Mapping of local directories to their corresponding remote directories
```

```
declare -A remote_directories=(
```

```
    ["IBFT"]="IBFT"
```

```
    ["LFT"]="LFT"
```

```
    ["SALARY_TRANSFER"]="Salary_tranfer"
```

```
    ["SMART_CHEQUE"]="Smart_cheque"
```

```
    ["CCHQ"]="CC01"
```

```
)
```

```
# Directory to store list of uploaded files
```

```
uploaded_files_dir="/opt/uploaded_files"
```

```
mkdir -p "$uploaded_files_dir"
```

```
# Function to upload files for a directory
upload_files() {
    local local_dir="$1"
    local remote_dir="$2"
    local uploaded_files_file="$uploaded_files_dir/$(basename "$local_dir").txt"
    touch "$uploaded_files_file"

    for file in "$local_dir"/*.txt; do
        # Skip if the file has already been uploaded
        if grep -Fxq "$(basename "$file")" "$uploaded_files_file"; then
            echo "File $(basename "$file") already uploaded."
            continue
        fi

        expect << EOF
log_user 1
spawn sftp $username@$host
expect {
    "Enter passphrase for key '/home/oracle/.ssh/id_rsa':" {
        send "$passphrase\r"
        expect "password:"
    }
}
```



```
}  
}  
expect "sftp>"  
send "lcd $local_dir\r"  
expect "sftp>"  
send "cd $remote_dir\r"  
expect "sftp>"  
send "put \"$(basename "$file")\" \r"  
expect {  
    "Uploading" {  
        exp_continue  
    }  
    "sftp>" {  
        send "exit\r"  
        expect eof  
        exit 0  
    }  
}  
default {  
    send_user "Upload of $(basename "$file") failed.\n"  
    send "exit\r"  
    expect eof  
    exit 1  
}
```

EOF

```
# If upload is successful, record the uploaded file
if [[ $? -eq 0 ]]; then
    echo "$(basename "$file")" >> "$uploaded_files_file"
else
    echo "Failed to upload $(basename "$file")"
fi
done
}

# Copy files to another local directory
copy_files() {
    local source_dir="$1"
    local destination_dir="$backup_root/$(basename "$source_dir")"
    mkdir -p "$destination_dir"
    cp -a "$source_dir"/*.txt "$destination_dir/"
}

# Watch for file creation or move events
inotifywait -r -m -e create -e moved_to --format '%w%f' "$local_root" |
while read -r file; do
    # Determine the directory of the file
```

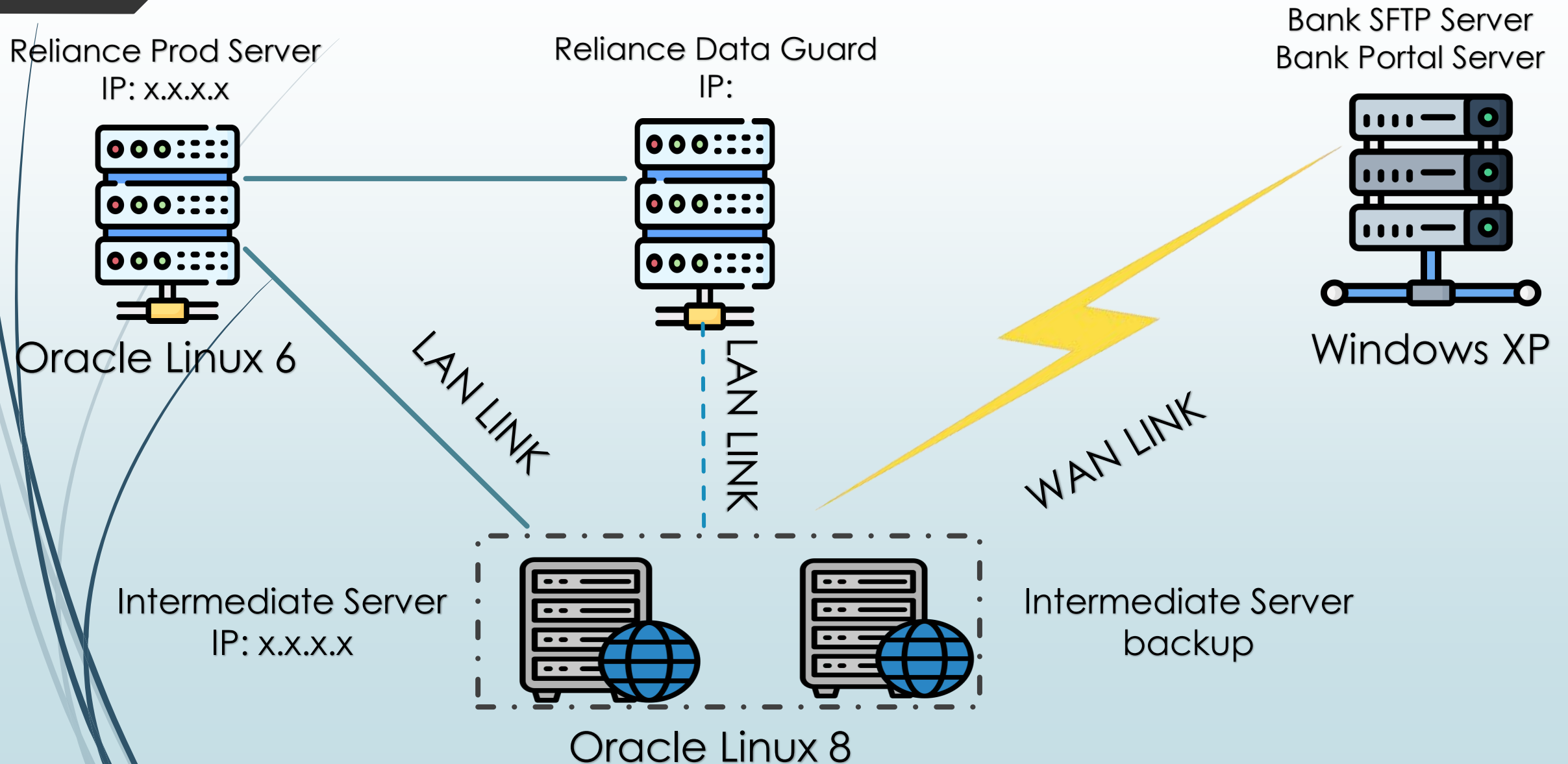
```
# Watch for file creation or move events
inotifywait -r -m -e create -e moved_to --format '%w%f' "$local_root" |
while read -r file; do
    # Determine the directory of the file
    file_dir=$(dirname "$file")
    # Iterate over directories and upload files if the file is in one of them
    for local_dir in "${!remote_directories[@]}"; do
        if [[ "$file_dir" == "$local_root/$local_dir" ]]; then
            remote_dir="${remote_directories[$local_dir]}"
            upload_files "$file_dir" "$remote_root/$remote_dir"
            copy_files "$file_dir" # Copy files to backup directory
            break
        fi
    done
done
```

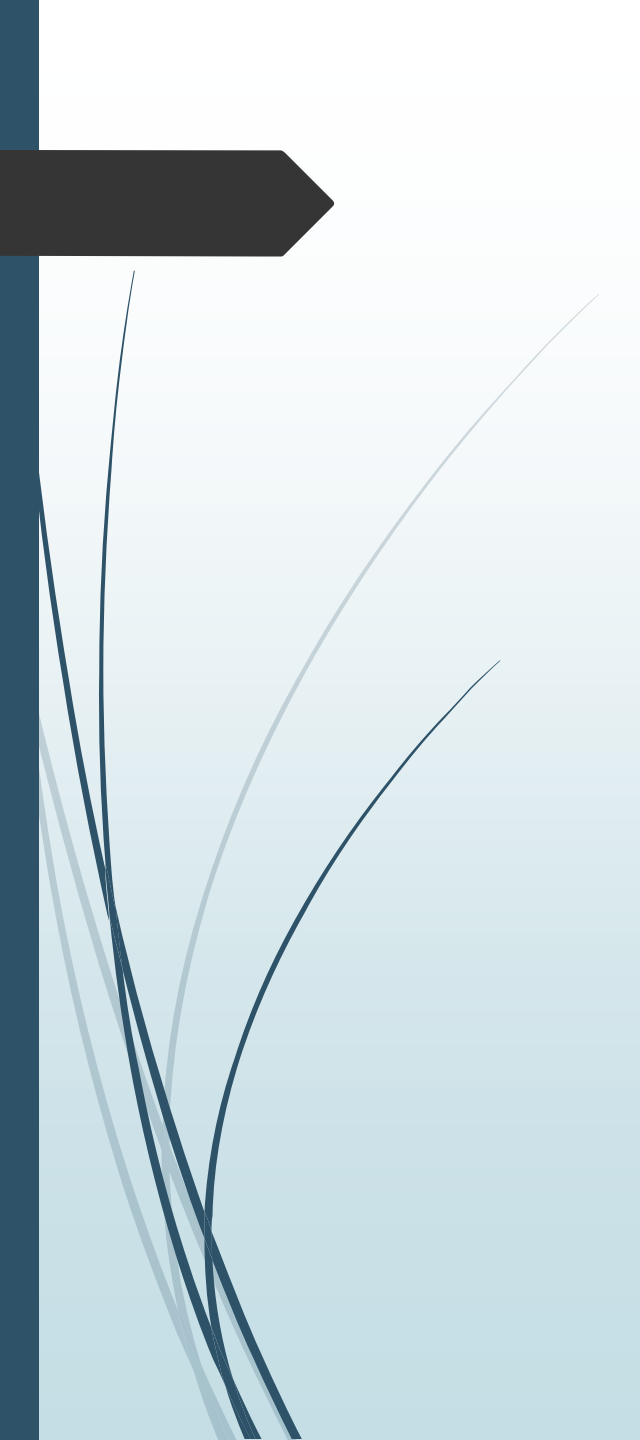


Requirements

- All this project is configured on an end of life refurbished server. As this is a real time activity and part of production cycle, 2 new mid range servers are required to avoid any delay in service delivery.
- We will keep one server in TPL and other at RWML data center. In case if one server goes down other replaces it.
- All the configurations are done on the Prod server and not on the Data Guard, during failure of Prod all bank activities will suffer. To avoid this case, configuration on Data Guard along with scripts placement is required. So that in case of disaster recovery mode, bank activities don't halt.

Secure & Robust Infrastructure





Thank You



Q/A Session

Suggestions